

How the ELF Epistemic Stack works

Brandon Flores brandon@brflores.com - ff epistack
claim. Not a slide deck!

Co-writing (straight)

This note was co-written with Guroor, the coding agent I work with daily. I call it
Brandon Flores's "vocalists" and have been collaborating with Guroor every day for
sixteen years of industrial design before that. If a line here is wrong, the error is

1. The problem (same bug as Jun 21 / Jul 19)

That's claiming can cite one brochure and still get counted as three independent s
citations. That's slipped a provenance genealogy layer: counting and repeat
this page could suggest authority from something that asks where the authority is
infrastructure (and evidence) support only! origins, LHC safety, or egg nutrition.

2. What the system does, in four stages

Four stages. Separate modules. Run in order.

Stage 1. Resolve identity (genealogy)

Code: src/epistemic/index.js, genealogy.js, source_identity.js

Level: a case folder (evidence_blocks.json, source_registry.json). Map every block to three

level: 1. Counts. Example quote in the JSON:

Lineage: One of the original observations. One experiment. One dataset, one generative event

lineage number always increase. Counting claims overstates how many excerpts you

have. Lineage is the conservative count. Overstating independence is the failure mode we

inflation factor printed on the report: excerpts documents lineages, with a citation

Stage 2. Propose

Code: src/epistemic/adiudicate.js (propose). It is

propositional assumption, and reasoning steps. Each step must cite a block and quote the passage

That output is a proposal. Not a verdict.

Stage 3. Verify (deterministic)

Code: verifyCitations() in adiudicate.js, measure.js

No model in this stage. For each reasoning step:

2: Text quotes must appear verbatim in the cited block.

3. Measurement blocks: re-run a whitelisted read-only command and compare declared value to measured value. Exact match
 Cite without quote flagged weak. All weak verdict unsubstantiated (not a pass).
 If a measurement can't run on this machine (repo not present), result is unverifiable_here.
 Third state. Not the same as failed. Unchecked never counts as verified.
 Stage 4 Challenge panel + verdict
 Code: mechanical_challenge.js, rest of adjudicate.js, model_identity.js
 Two seats on the panel.
 Seat 1: mechanical challenger. Always runs. Zero models. Eight checks (C1-C8); coverage
 and coverage can be matched. The model independence is a display of code, not a
 calibration. This is the only seat whose independence we can guarantee.
 Seat 2: model challenger (optional). Sees the conclusion, not the reasoning (blind). Picked
 from the proposers. Same family = second ear; Not second source. If is verified different
 Verdicts (computed, not declared):
 Verdict: 1. Meaningful
 Unsubstantiated check failed. Done
 Verified - unchecked. Challenge counts and lineage ran
 contested with two or more independent lineages objected.
 Human is the last lineage on the panel. Recording a position raises independence to strong.
 Positions write to decisions/.json. On the record is real, not copy.

3. How that answers the bug
 Overcounting. Stage 1 reports lineages, not citation counts. Cochrane's Handbook (Ch. 5)

still says there is no fully automated recommended tool for reconciling multiple reports of
in these workflows. I haven't invented that problem. We shipped tooling for the full that stays manual
unchallenged conclusions. Stages 2-4 force shown work, mechanical citation checks, then a
any reference is documented (Panickssery et al., NeurIPS 2024). Search is the structural
Self-check. Case self-points the tool at claims about me. Mechanical challenger catches
every statement in my own evidence base, mapped as one. I made a linked several times in COVID.
Verify: register fails if a claimed guard is missing. Every entry names a test. npm run

4. What it does / doesn't (honest)

Does: solve claim, document, lineage and print the conservative count
Does: measure mechanical challenge every time
Does: papers (Elicit)
Does: map elements at web scale (Scite)
Does: some the underlying scientific questions at corpus scale (CiteAudit, CiteTracer, and related tools)
Does: claim stays narrow: structure and checkable independence, not better prose and not
domain expertise.

5. What's in the repo now (verify, don't trust me)

As of this build: npm test 188 passing in the epistack repo (no network). Re-run before
review.

Caveat: Where OVER NOTE TO ELF.md
 Based on: https://github.com/doctrinelabs/epistack/blob/master/docs/README.md
 Mechanical checks C1-C8 : • src/epistemic/mechanical_challenge.js
 In passing: https://github.com/brandon255/doctrine-labs-elf-epistack.git
 All doctrine-labs/epistack
 All fun epistemic: covid

No install beyond Node 20. No API key. Mechanical half runs cold. Local model optional for live proposer/challenge. Prefer not to clone, open docs/transcripts/ on GitHub.

6. What I claim / what I don't

What I claim: I found a real gap while dogfooding, built the navigation layer in the early package, then I found the assessment layer on top. It runs. Tests pass. Failures are named.

What I do not claim: first to notice the problem (Greenberg 2009; Cochrane handbook). Senior engineer credentials. That every adjacent tool is worse. That COVID/LHC/eggs are settled.

I'm not submitting as an epistemologist. I'm submitting as a builder who found an infrastructure gap while doing PLP homework, then kept building.

Brandon

Doctrine Labs • co-written with Cursor (Rivet) • Brandon Flores